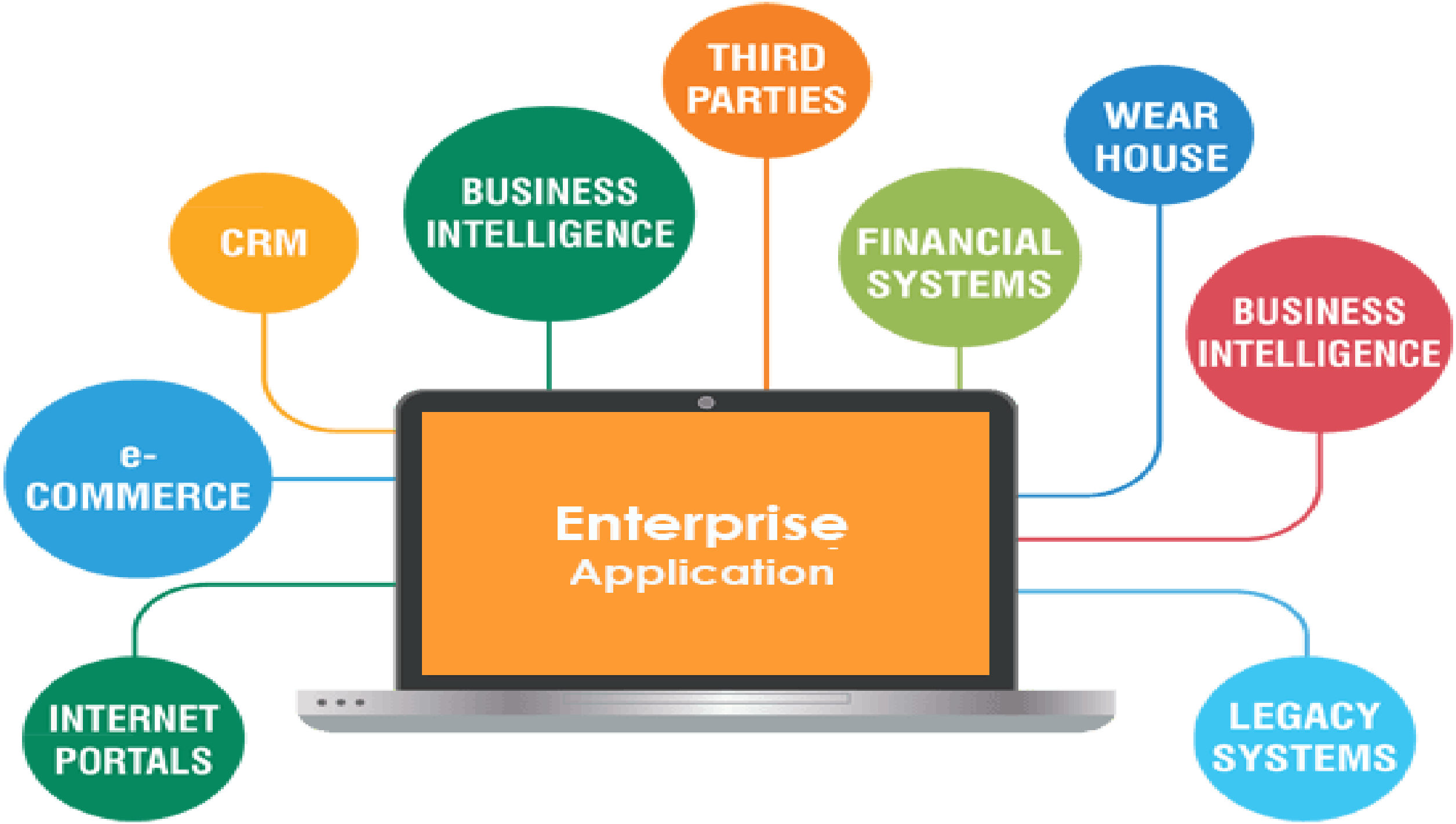
COURSE DETAILS

Enterprise Architecture


THIRD PARTIES
WEAR HOUSE
BUSINESS INTELLIGENCE
CRM
FINANCIAL SYSTEMS
BUSINESS INTELLIGENCE
e-COMMERCE
Enterprise Application
INTERNET PORTALS
LEGACY SYSTEMS

Good Design

- Correctness
- Maintainability & Modifiability
- Reusability
- Simplicity & Readability
- Performance & Scalability
- Testability

SOLID Principles

- SOLID is a set of five design principles for object-oriented software.
- Goal: keep codebases flexible, maintainable, and easier to change safely.

Single Responsibility Principle (SRP)

- A class or module should have only one reason to change.
- Keeps responsibilities focused and reduces side effects when requirements change.
- Improves readability, testability, and reuse of components.
- Typical signs of violation: 'God classes' handling UI, domain logic, and persistence together.

Violation of SRP example

```
public class UserService
{
    public void Register(string email, string password)
    {
        if (!ValidateEmail(email))
            throw new ValidationException("Email is not an email");
            var user = new User(email, password);


SendEmail(new MailMessage("mysite@nowhere.com", email) { Subject="HEl
    }
    public virtual bool ValidateEmail(string email)
    {
      return email.Contains("@");
    }
    public bool SendEmail(MailMessage message)
    {
      _smtpClient.Send(message);
    }
}
```

Violation of SRP Result

- Tight Coupling : Binds two (or more) details together in a way that's difficult to change.

- Separation of Concerns : Programs should be separated into distinct sections, each addressing a separate concern

Open/Closed Principle (OCP)

- Software entities should be open for extension but closed for modification.
- New behavior is added by extending existing types, not by modifying stable code.
- Reduces risk of breaking working features when adding new variants.
- Often implemented via interfaces, inheritance, strategies, or composition.

Violation of OCP example

```
0 references
public class ReportGeneration
{
    2 references
    public string ReportType { get; set; }

    0 references
    public void GenerateReport(object data)
    {
        if (ReportType == "CRS")
        {
            // Report generation with employee data in Crystal Report.
        }
        if (ReportType == "PDF")
        {
            // Report generation with employee data in PDF.
        }
    }
}
```

Fool me once (take the first bullet)

Fool me once, shame on you. Fool me twice, shame on me. This means that we initially write our code expecting it not to change. When a change occurs, we implement the abstractions that protect us from future changes of that kind.

Liskov Substitution Principle (LSP)

- Subtypes must be substitutable for their base types without breaking correctness.
- Clients depending on the base type should work with any valid subtype.
- Violations appear when subtypes weaken guarantees or throw unexpected exceptions.
- Good subtype design preserves the behavioral contract of the base type.

Violation of LSP In C#

```
ICollection<T> : IEnumerable<T>
```

Defines:

- Add
- Clear
- Contains
- CopyTo
- Remove

```
ReadOnlyCollection<T> : ICollection<T>
```

- Add

Interface Segregation Principle (ISP)

- Clients should not be forced to depend on interfaces they do not use.
- Prefer many small, focused interfaces over single 'fat' ones.
- Reduces coupling and makes implementations and tests simpler.
- Encourages designing contracts around cohesive responsibilities.

Violation of ISP example

```
public abstract class MembershipProvider : ProviderBase
{
    // Fields
    private MembershipValidatePasswordEventHandler _EventHandler;
    private const int SALT_SIZE_IN_BYTES = 0x10;

    // Events
    public event MembershipValidatePasswordEventHandler ValidatingPassword;

    // Methods
    protected MembershipProvider();
    public abstract bool ChangePassword(string username, string oldPassword, string newPassword);
    public abstract bool ChangePasswordQuestionAndAnswer(string username, string password, string newPasswordQuestion, string newPasswordAnswer);
    public abstract MembershipUser CreateUser(string username, string password, string email, string passwordQuestion, string passwordAnswer, bool isApproved, object providerUser
    protected virtual byte[] DecryptPassword(byte[] encodedPassword);
    public abstract bool DeleteUser(string username, bool deleteAllRelatedData);
    internal string EncodePassword(string pass, int passwordFormat, string salt);
    protected virtual byte[] EncryptPassword(byte[] password);
    public abstract MembershipUserCollection FindUsersByEmail(string emailToMatch, int pageIndex, int pageSize, out int totalRecords);
    public abstract MembershipUserCollection FindUsersByName(string usernameToMatch, int pageIndex, int pageSize, out int totalRecords);
    internal string GenerateSalt();
    public abstract MembershipUserCollection GetAllUsers(int pageIndex, int pageSize, out int totalRecords);
    public abstract int GetNumberOfUsersOnline();
    public abstract string GetPassword(string username, string answer);
    public abstract MembershipUser GetUser(object providerUserKey, bool userIsOnline);
    public abstract MembershipUser GetUser(string username, bool userIsOnline);
    internal MembershipUser GetUser(string username, bool userIsOnline, bool throwOnError);
    public abstract string GetUserNameByEmail(string email);
    protected virtual void OnValidatingPassword(ValidatePasswordEventArgs e);
    public abstract string ResetPassword(string username, string answer);
    internal string UnEncodePassword(string pass, int passwordFormat);
    public abstract bool UnlockUser(string userName);
    public abstract void UpdateUser(MembershipUser user);
    public abstract bool ValidateUser(string username, string password);

    // Properties
    public abstract string ApplicationName { get; set; }
    public abstract bool EnablePasswordReset { get; }
    public abstract bool EnablePasswordRetrieval { get; }
    public abstract int MaxInvalidPasswordAttempts { get; }
    public abstract int MinRequiredNonAlphanumericCharacters { get; }
    public abstract int MinRequiredPasswordLength { get; }
    public abstract int PasswordAttemptWindow { get; }
    public abstract MembershipPasswordFormat PasswordFormat { get; }
    public abstract string PasswordStrengthRegularExpression { get; }
    public abstract bool RequiresQuestionAndAnswer { get; }
```

ASP.NET MembershipProvider class – a very “fat” interface!

Activate
Go to Settin

Dependency Inversion Principle (DIP)

- High-level modules should not depend on low-level modules; both depend on abstractions.
- Abstractions should not depend on details; details depend on abstractions.

Violation of DIP example

```
1 reference
public class AccountService
{
    private AccountRepository repository;
    0 references
    public AccountService()
    {
        repository = new AccountRepository();
    }
    0 references
    public void Withraw(decimal amount, int accountId)
    {
        var account = repository.Get(accountId);

        account.Withraw(amount);

        repository.Update(account);
    }

}
```

Dependency Injection

- Dependency Injection is a technique that is used to allow calling code to inject the dependencies a class needs when it is instantiated .

What is Enterprise Software?

- Enterprise applications are about the display, manipulation, and storage of large amounts of often complex data and the support or automation of business processes with that data

What is Enterprise Software?

- Enterprise applications are complex, scalable,
- distributed, component-based.
- In short, they are highly complex systems.

Maintainability, Extensibility Is Important

Software Design

Software design is the process of defining software solutions to one or more sets of problems

Characteristics of a good design

- High Cohesion . All responsibilites of a module should be related .

- Low Coupling .How much a module depends upon others

Code Smells

- A smell is not automatically a bug, but a sign that design may fail under change.
- Smells are early warning signals that the design is drifting in a dangerous direction.

Rigidity

- Rigidity: the system is hard to change.
- A simple change requires modifications in many modules.
- Developers are afraid to touch code because everything feels connected.
- Symptoms: long change cycles, frequent merge conflicts, high risk of side effects.
- Typical remedies: apply SRP/OCP, refactor towards clear modules and boundaries.

Fragility

- Fragility: changes cause the system to break in unexpected places.
- You modify one feature and unrelated features start failing.
- Indicates hidden coupling and poor encapsulation.
- Symptoms: many regression bugs after each release.
- Typical remedies: improve encapsulation, write regression tests, reduce hidden dependencies.

Immobility

- Immobility: hard to reuse parts of the system in other contexts.
- Useful code is entangled with unrelated responsibilities or frameworks.
- Developers copy-and-paste instead of reusing components.
- Symptoms: multiple similar implementations across codebase.
- Typical remedies: extract reusable components, introduce clear interfaces and modules.

This is an unfortunate but very common occurrence.

Needless Complexity

- Needless Complexity: design contains infrastructure or abstractions that are not needed.
- Speculative features, extra layers, and patterns that do not solve real problems.
- Increases learning curve and maintenance overhead for no benefit.
- Symptoms: complicated designs with little business value.

YAGNI - YOU AREN'T GONA NEED IT

Needless Repetition

- Needless Repetition: the same ideas are implemented in multiple ways.
- Not just duplicated code, but duplicated concepts and patterns.
- Makes the system harder to understand and evolve consistently.
- Symptoms: similar logic scattered with small variations.
- Typical remedies: consolidate implementations, introduce shared abstractions or services.

This is an unfortunate but very common occurrence.

Opacity

It's hard to read and undrestand.it does not express it's intent well

Viscosity

- Doing thing right is harder than doing thions wrong

- Viscosity of Software Some of the ways preserve the design; others do not (i.e., they are hacks). hen the design-preserving methods are more difficult to use than the hacks, the viscosity of the design is high.

- Viscosity of environment comes about when the development environment is slow and inefficient. For example, if compile times are very long, developers will be tempted to make changes that don't force large recompiles, even though those changes don't preserve the design

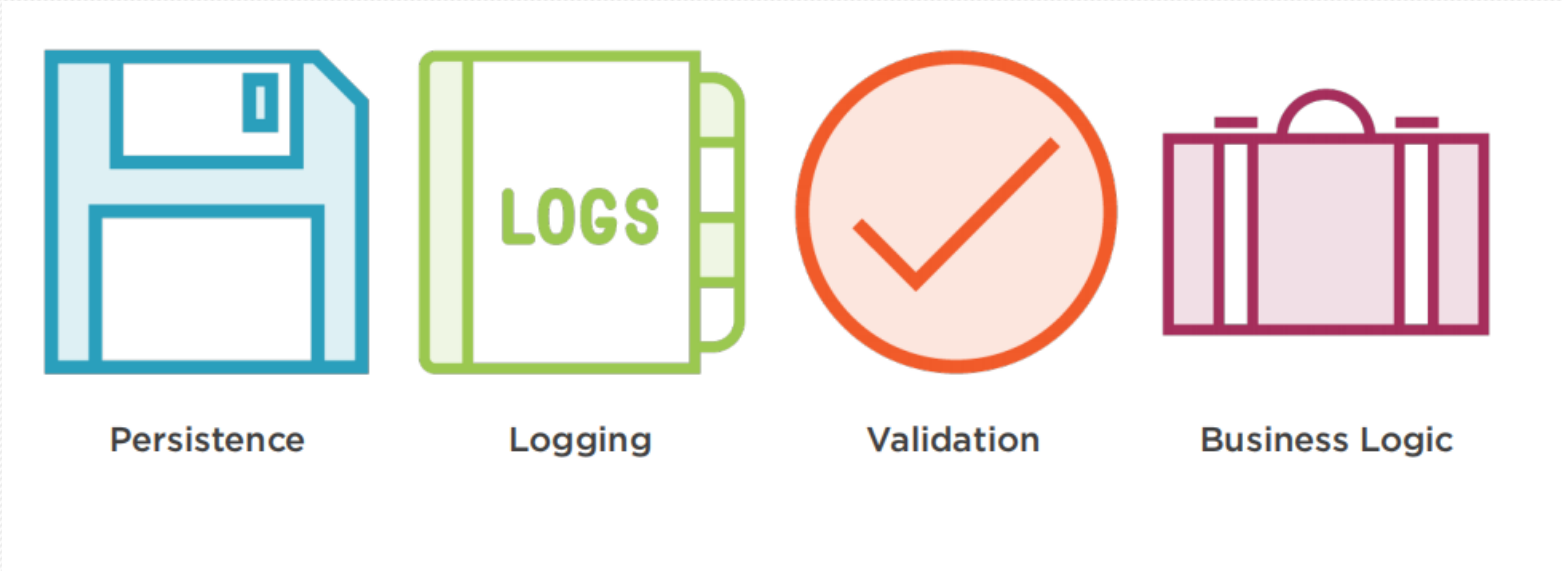
SOLID Principles

- SOLID is a set of five design principles for object-oriented software.
- Goal: keep codebases flexible, maintainable, and easier to change safely.
- They guide responsibility placement and reduce unnecessary coupling.
- Strongly support rich domain models and DDD-style design.

Single Responsibility Principle (SRP)

- A class or module should have only one reason to change.
- Keeps responsibilities focused and reduces side effects when requirements change.
- Improves readability, testability, and reuse of components.
- Typical signs of violation: 'God classes' handling UI, domain logic, and persistence together.

What Is a Responsibility?



Violation of SRP example

```
public class UserService
{
    public void Register(string email, string password)
    {
        if (!ValidateEmail(email))
            throw new ValidationException("Email is not an email");
            var user = new User(email, password);


SendEmail(new MailMessage("mysite@nowhere.com", email) { Subject="HEl
    }
    public virtual bool ValidateEmail(string email)
    {
      return email.Contains("@");
    }
    public bool SendEmail(MailMessage message)
    {
      _smtpClient.Send(message);
    }
}
```

Violation of SRP Result

- Tight Coupling : Binds two (or more) details together in a way that's difficult to change.

- Separation of Concerns : Programs should be separated into distinct sections, each addressing a separate concern

Open/Closed Principle (OCP)

- Software entities should be open for extension but closed for modification.
- New behavior is added by extending existing types, not by modifying stable code.
- Reduces risk of breaking working features when adding new variants.
- Often implemented via interfaces, inheritance, strategies, or composition.

Violation of OCP example

```
0 references
public class ReportGeneration
{
    2 references
    public string ReportType { get; set; }

    0 references
    public void GenerateReport(object data)
    {
        if (ReportType == "CRS")
        {
            // Report generation with employee data in Crystal Report.
        }
        if (ReportType == "PDF")
        {
            // Report generation with employee data in PDF.
        }
    }
}
```

How modify a module without changing its source code?

The answer is Abstraction.

Fool me once (take the first bullet)

Fool me once, shame on you. Fool me twice, shame on me.This means that we initially write our code expecting it not to change. When a change occurs, we implement the abstractions that protect us from future changes of that kind.

Stimulating change

If we decide to take the first bullet, it is to our advantage to get the bullets early and frequently.

- Test-First approach.
- Use very short development cycles.
- Develop features before infrastructure and show those features to stakeholders.

Liskov Substitution Principle (LSP)

- Subtypes must be substitutable for their base types without breaking correctness.
- Clients depending on the base type should work with any valid subtype.
- Violations appear when subtypes weaken guarantees or throw unexpected exceptions.
- Good subtype design preserves the behavioral contract of the base type.

Violation of LSP example

```
public class Rectangle
{
    private double width;
    private double height;
    6 references
    public virtual double Width
    {
        get { return width; }
        set { width = value; }
    }
    6 references
    public virtual double Height
    {
        get { return height; }
        set { height = value; }
    }

    2 references | ✖ 1/2 passing
    public double Area()
    {
        return Height * Width;
    }
}
1 reference
```

```
public class Square : Rectangle
{
    6 references
    public override double Width
    {
        set
        {
            base.Width = value;
            base.Height = value;
        }
    }
    6 references
    public override double Height
    {
        set
        {
            base.Height = value;
            base.Width = value;
        }
    }
}
```

Violation of LSP example

```
Rectangle rect = new Rectangle() { Width = 2, Height = 5 };
int rectArea = AreaCalculator.CalcArea(rect);
Assert.AreEqual((rect.Width * rect.Height), rectArea);

Rectangle square = new Square() { Height = 2, Width = 10 };
int squareArea = AreaCalculator.CalcArea(square);
Assert.AreEqual((rect.Width * rect.Height), rectArea);
```

Violation of LSP In C#

```
ICollection<T> : IEnumerable<T>
```

Defines:

- Add
- Clear
- Contains
- CopyTo
- Remove

```
ReadOnlyCollection<T> : ICollection<T>
```

- Add

Common Smells

- Method throws NotSupportedException Or Empty implementations
- Switch-case statements with checks of the actual type of an argument(Downcasts).

Interface Segregation Principle (ISP)

- Clients should not be forced to depend on interfaces they do not use.
- Prefer many small, focused interfaces over single 'fat' ones.
- Reduces coupling and makes implementations and tests simpler.
- Encourages designing contracts around cohesive responsibilities.

Violation of ISP example

```
public abstract class MembershipProvider : ProviderBase
{
    // Fields
    private MembershipValidatePasswordEventHandler _EventHandler;
    private const int SALT_SIZE_IN_BYTES = 0x10;

    // Events
    public event MembershipValidatePasswordEventHandler ValidatingPassword;

    // Methods
    protected MembershipProvider();
    public abstract bool ChangePassword(string username, string oldPassword, string newPassword);
    public abstract bool ChangePasswordQuestionAndAnswer(string username, string password, string newPasswordQuestion, string newPasswordAnswer);
    public abstract MembershipUser CreateUser(string username, string password, string email, string passwordQuestion, string passwordAnswer, bool isApproved, object providerUser
    protected virtual byte[] DecryptPassword(byte[] encodedPassword);
    public abstract bool DeleteUser(string username, bool deleteAllRelatedData);
    internal string EncodePassword(string pass, int passwordFormat, string salt);
    protected virtual byte[] EncryptPassword(byte[] password);
    public abstract MembershipUserCollection FindUsersByEmail(string emailToMatch, int pageIndex, int pageSize, out int totalRecords);
    public abstract MembershipUserCollection FindUsersByName(string usernameToMatch, int pageIndex, int pageSize, out int totalRecords);
    internal string GenerateSalt();
    public abstract MembershipUserCollection GetAllUsers(int pageIndex, int pageSize, out int totalRecords);
    public abstract int GetNumberOfUsersOnline();
    public abstract string GetPassword(string username, string answer);
    public abstract MembershipUser GetUser(object providerUserKey, bool userIsOnline);
    public abstract MembershipUser GetUser(string username, bool userIsOnline);
    internal MembershipUser GetUser(string username, bool userIsOnline, bool throwOnError);
    public abstract string GetUserNameByEmail(string email);
    protected virtual void OnValidatingPassword(ValidatePasswordEventArgs e);
    public abstract string ResetPassword(string username, string answer);
    internal string UnEncodePassword(string pass, int passwordFormat);
    public abstract bool UnlockUser(string userName);
    public abstract void UpdateUser(MembershipUser user);
    public abstract bool ValidateUser(string username, string password);

    // Properties
    public abstract string ApplicationName { get; set; }
    public abstract bool EnablePasswordReset { get; }
    public abstract bool EnablePasswordRetrieval { get; }
    public abstract int MaxInvalidPasswordAttempts { get; }
    public abstract int MinRequiredNonAlphanumericCharacters { get; }
    public abstract int MinRequiredPasswordLength { get; }
    public abstract int PasswordAttemptWindow { get; }
    public abstract MembershipPasswordFormat PasswordFormat { get; }
    public abstract string PasswordStrengthRegularExpression { get; }
    public abstract bool RequiresQuestionAndAnswer { get; }
```

ASP.NET MembershipProvider class – a very “fat” interface!

Activate
Go to Settin

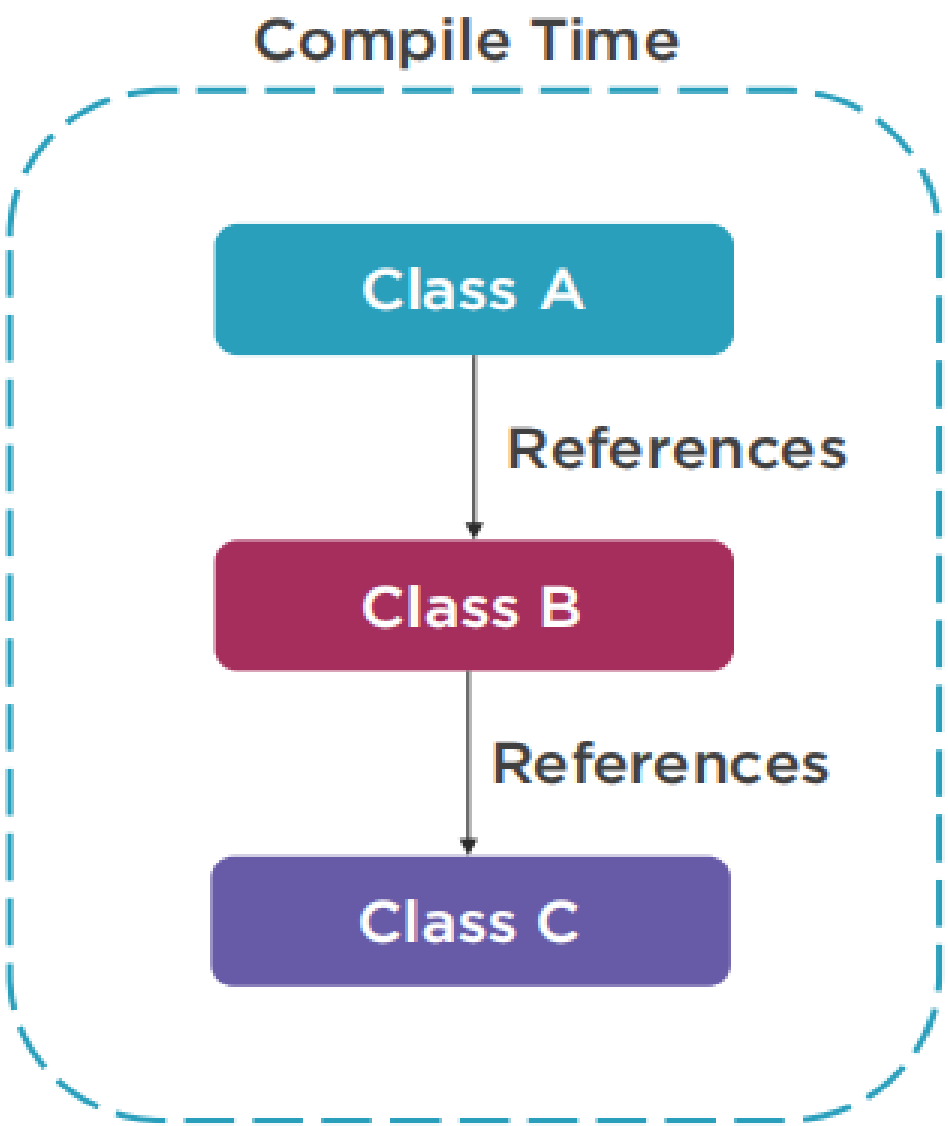
ISP Violation Problems

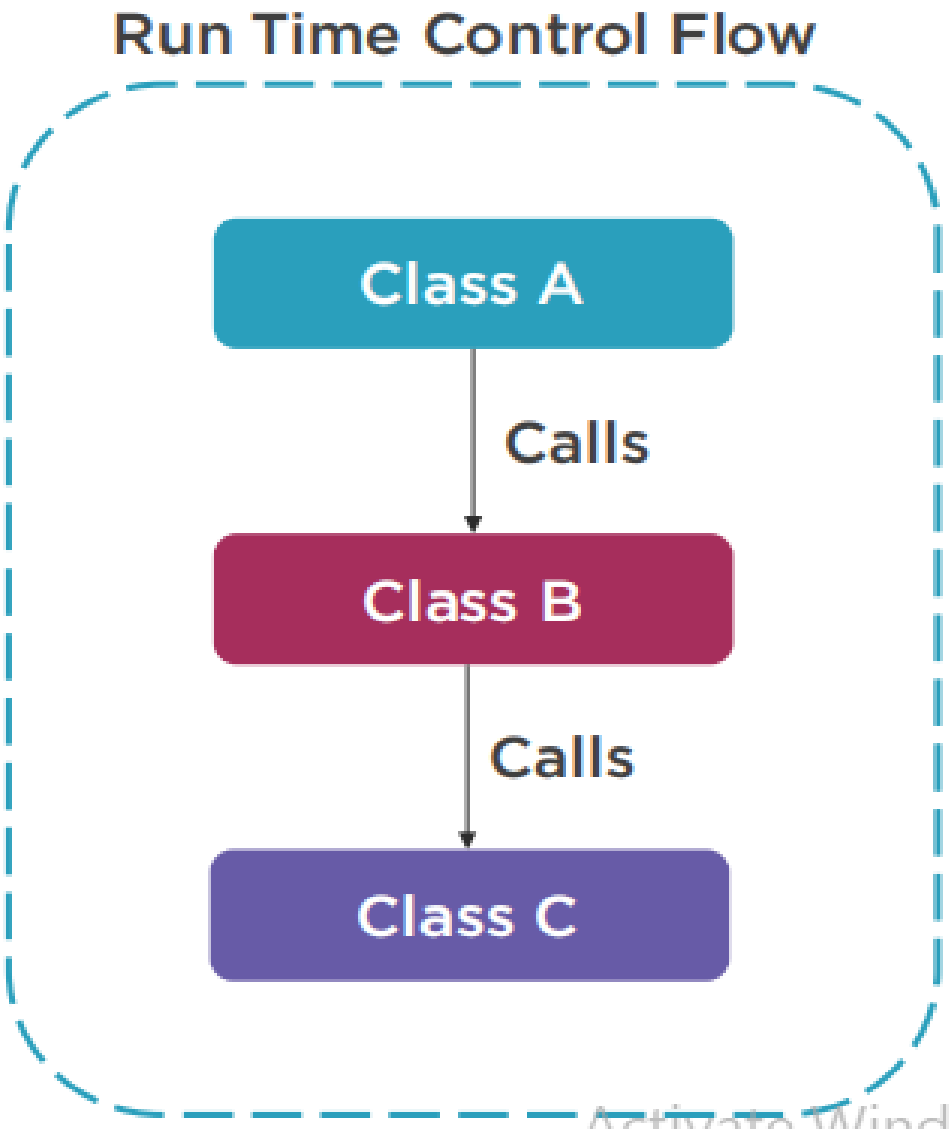
- Violate LSP
- Violate SRP , More coupling
- Hard to test

Dependency Inversion Principle (DIP)

- High-level modules should not depend on low-level modules; both depend on abstractions.
- Abstractions should not depend on details; details depend on abstractions.
- Supports hexagonal/clean architectures where domain depends only on ports (interfaces).
- Enables swapping infrastructure (DB, messaging, UI) without changing domain code.

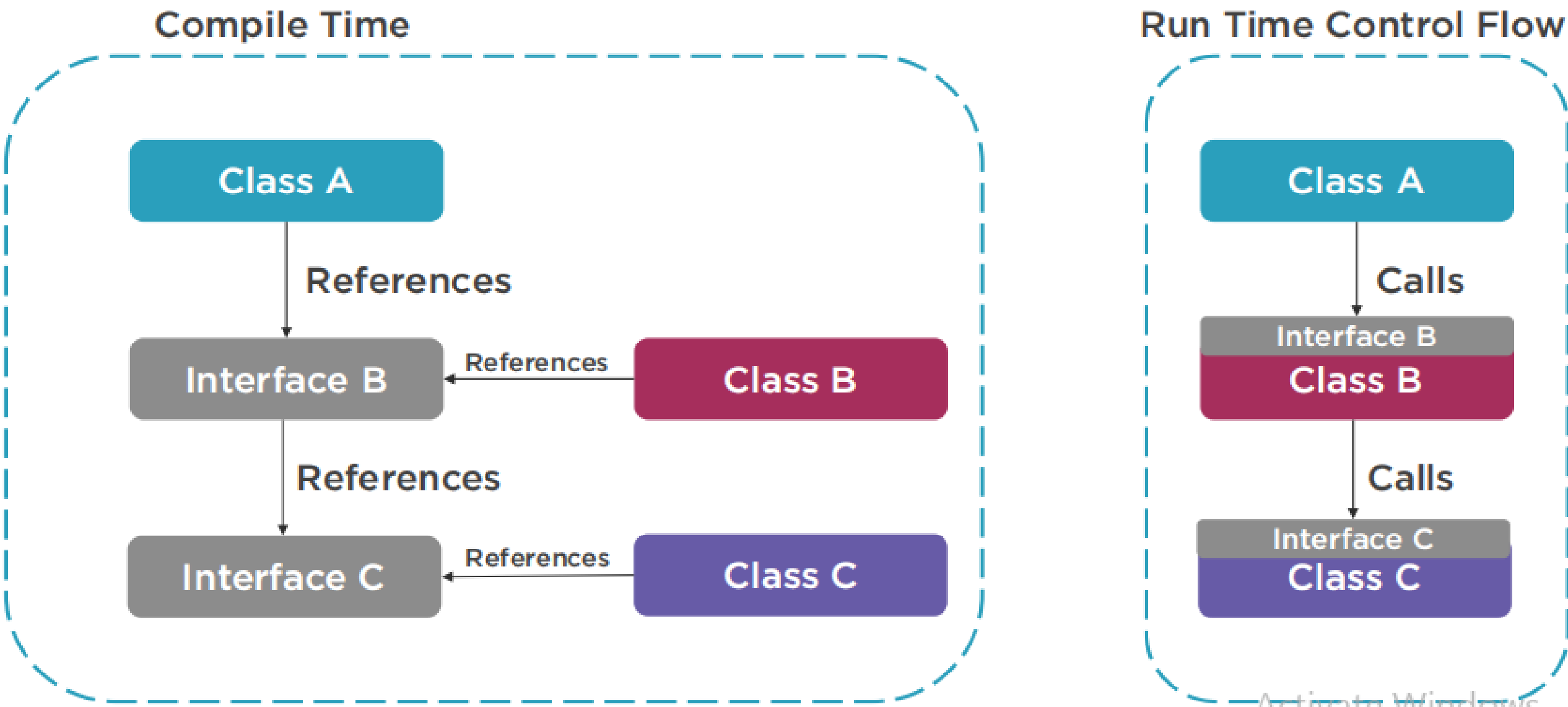
Dependencies without Abstractions





Activate Windows

Dependencies with Abstractions



Violation of DIP example

```
1 reference
public class AccountService
{
    private AccountRepository repository;
    0 references
    public AccountService()
    {
        repository = new AccountRepository();
    }
    0 references
    public void Withraw(decimal amount, int accountId)
    {
        var account = repository.Get(accountId);

        account.Withraw(amount);

        repository.Update(account);
    }

}
```

DIP Violation Problems

- Coupling ↑
- Cohission ↓
- Testability ↓

Dependency Injection

- Dependency Injection is a technique that is used to allow calling code to inject the dependencies a class needs when it is instantiated .

Dependency Injection Forms

- Constructor Injection
- Property Injection
- Method Injection